VELLORE INSTITUTE OF TECHNOLOGY VELLORE

School of Computer Science and Engineering (SCOPE)

MINI PROJECT REPORT

QureFlow Clinic Queue and Appointment Management Using MySQL

BCSE302P Database Systems Laboratory

Submitted by

S. No.	Name	Registration Number
1	Kartikay Gupta	24BCI0109
2	Sreyanko Sinha	24BCB0059

Under the guidance of

Dr. G. M. Karthik

Associate Professor, School of Computer Science and Engineering (SCOPE)

Vellore Institute of Technology, Vellore

Date of Submission: 19 October 2026

CERTIFICATE

This is to certify that the Mini Project Report titled QureFlow Clinic Queue and Appointment Management Using MySQL submitted by Kartikay Gupta (24BCI0109) and Sreyanko Sinha (24BCB0059), in partial fulfilment of the requirements of BCSE302P Database Systems Laboratory, is a record of their project work during the Fall Semester 2026-27.

Declaration of originality

We declare that this report describes our QureFlow project and its locally executed MySQL demonstration. The test identities and clinical organisations are synthetic. External technical documentation is acknowledged in the references, and execution results are distinguished from features requiring further device verification.

Kartikay Gupta | 24BCI0109

Sreyanko Sinha | 24BCB0059

Signature of the Faculty

Dr. G. M. Karthik

Associate Professor, SCOPE

VIT, Vellore

Place: Vellore

Date: 19 October 2026

ABSTRACT

QureFlow is a clinic queue and appointment management application designed to make visit planning and queue progression more transparent. This project presents its MySQL database variant for the Database Systems Laboratory. The implementation separates clinics, doctors, specialties, languages, users, appointments and queue tokens into relational tables with explicit keys and constraints. SQL joins, filters, ordering and pagination support doctor selection, while InnoDB transactions and row locks protect appointment capacity and concurrent queue operations. A stored function measures consultation duration, a procedure produces daily patient-flow results, three triggers enforce and record queue transitions, and an explicit cursor returns doctor workload. A React web interface communicates with an Express API through parameterised database queries. The same interface is packaged as an Android application, with local server discovery and anonymous connection verification. Testing covers valid workflows, invalid transitions, duplicate visits, clinic isolation, injection inputs and simultaneous bookings. All 24 MySQL integration cases passed in the recorded run, and browser booking and cancellation were verified against SQL records. The result is a reproducible database demonstration; production deployment and final hotspot acceptance on Android remain future verification tasks.

Keywords: MySQL, queue management, normalization, transactions, triggers, Android

TABLE OF CONTENTS

CERTIFICATE 2

ABSTRACT 3

1 INTRODUCTION 6

1.1 Problem Statement 6

1.2 Objectives 6

1.3 Importance of DBMS 6

1.4 Scope of the Project 7

2 SYSTEM REQUIREMENTS 8

2.1 Hardware Requirements 8

2.2 Software Requirements 8

3 DATABASE DESIGN 9

3.1 Entity Relationship Diagrams 9

3.2 Relational Schema 11

3.3 Schema Definition and Constraints 17

3.4 Normalization 18

4 IMPLEMENTATION 20

4.1 Functions 20

4.2 Procedures 21

4.3 Packages 22

4.4 Triggers 23

4.5 Cursors 24

4.6 SQL Search and Transaction Control 26

5 FRONT END AND ANDROID INTEGRATION 28

5.1 User Interface 28

5.2 Integration with the DBMS 32

5.3 Android Application 33

6 VERIFICATION AND TESTING 34
6.1 Test Cases 34
6.2 Output Screenshots 37
6.3 Validation of Functionality 38
7 RESULTS AND DISCUSSION 39
7.1 Achievements 39
7.2 Challenges Faced 39
7.3 Possible Improvements 39
8 CONCLUSION 40
8.1 Summary of Learning Outcomes 40
8.2 Future Scope 40
9 REFERENCES 41
10 APPENDICES 42
Appendix A Complete Schema Definition 42
Appendix B Additional Code and Source Map 47
Appendix C Terminal Demonstration 49
Appendix D Evidence Provenance 49

1 INTRODUCTION

1.1 Problem Statement

A manual clinic queue gives patients little visibility into their position and gives reception staff limited protection against duplicate entries or conflicting calls. Appointment records, doctor schedules and queue state can also diverge when they are maintained in separate spreadsheets. QureFlow brings these records into one database-backed workflow so that a patient can select a doctor and book a visit, while staff manage the corresponding queue.

The objective is to reduce avoidable uncertainty and waiting through better scheduling and queue visibility. The recorded experiments verify data integrity and workflow behaviour; they do not constitute a clinical study or quantify a reduction in real hospital waiting time.

1.2 Objectives

1. Design relational master data for clinics, doctors, specialties, languages and users.

2. Perform doctor selection, filtering, ordering and pagination through SQL queries.

3. Create appointments and tokens atomically while protecting slot capacity under concurrent requests.

4. Enforce queue transitions and audit changes with database constraints and triggers.

5. Integrate patient and staff web screens with the MySQL API and package the interface for Android.

6. Verify database objects and negative cases through repeatable tests and captured execution output.

1.3 Importance of DBMS

Primary and foreign keys connect records without storing the whole application as document blobs. Unique indexes prevent duplicate identities and conflicting active visits. Transactions let a booking either create all linked records or roll back. Row locks serialize competing updates to a doctor or appointment slot. Parameterised queries keep input values separate from SQL syntax, and application authorization restricts records to the permitted user and clinic. These controls address different failure modes and are used together.

1.4 Scope of the Project

The scope includes a local MySQL 8.4 database, patient and staff authentication for synthetic accounts, a doctor directory, scheduled appointments, queues, cancellation, audit records, daily analytics and an Android debug package. The earlier AWS DynamoDB/Cognito variant is retained separately; it is not the database or authentication path evaluated in this report.

Clinical diagnosis, automatic medical triage, billing, electronic health-record interoperability and production hospital deployment are outside this laboratory submission. Consultation estimates are operational guidance. Actual Android hotspot discovery remains a device acceptance item.

2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The local execution workstation uses Windows 11 and 16 GB RAM. The following is a practical development recommendation, not a measured minimum specification.

Component	Specification
Processor	64-bit processor with virtualization enabled for Docker Desktop
Memory	8 GB or more recommended; 16 GB used for the recorded run
Storage	At least 10 GB free for dependencies, database volume and build tools
Network	Local Wi-Fi or phone hotspot for Android-to-computer testing
Optional device	Android API 24 or newer for installing the debug APK

2.2 Software Requirements

Category	Tool or version
Database	MySQL 8.4.11 with InnoDB, in the mysql:8.4 Docker image
Database client	MySQL command-line client inside the container; Workbench is optional
Backend	Node.js, TypeScript, Express 4 and mysql2/promise
Web interface	React 18, TypeScript 5, Vite 6 and Tailwind CSS
Authentication	Local synthetic accounts, bcrypt password hashes and JWT sessions
Android	Capacitor 8.5.3, SDK API 36, Gradle 8.14.3 and JDK 22
Verification	Node test runner, TypeScript execution and browser automation
Local endpoints	Web: localhost:5174; API: port 5001; MySQL: 127.0.0.1:3307

3 DATABASE DESIGN

3.1 Entity Relationship Diagrams

The database contains 18 base tables and four views. The diagrams divide the model into catalog data, visit data and operational state to keep names and cardinalities readable. Arrows follow parent-to-child relationships. A 1:N label permits zero or more children for a parent; a non-null foreign key requires each child to reference one parent.

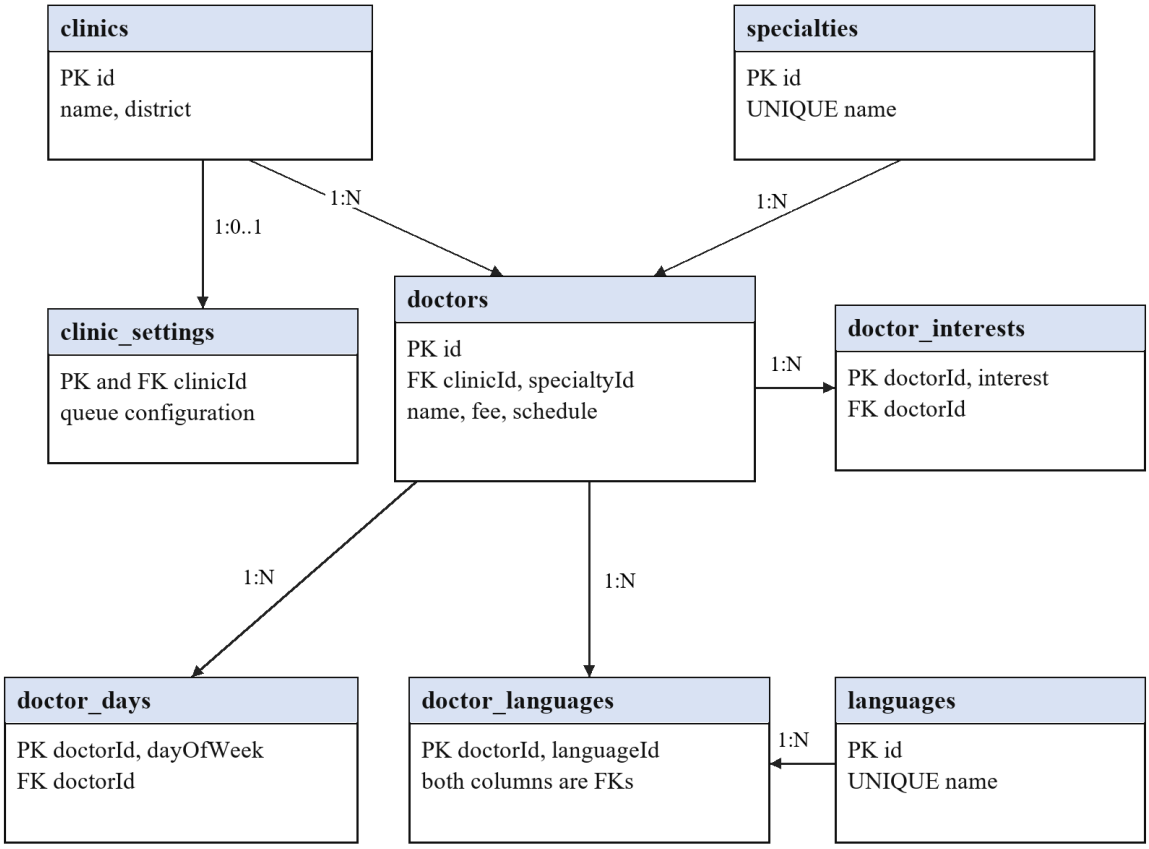


Figure 3.1 Catalog relations and doctor membership tables

Every doctor belongs to one clinic and one specialty. Languages form a many-to-many relationship through doctor_languages. The doctor_days and doctor_interests tables remove repeating lists. A clinic has at most one settings row.

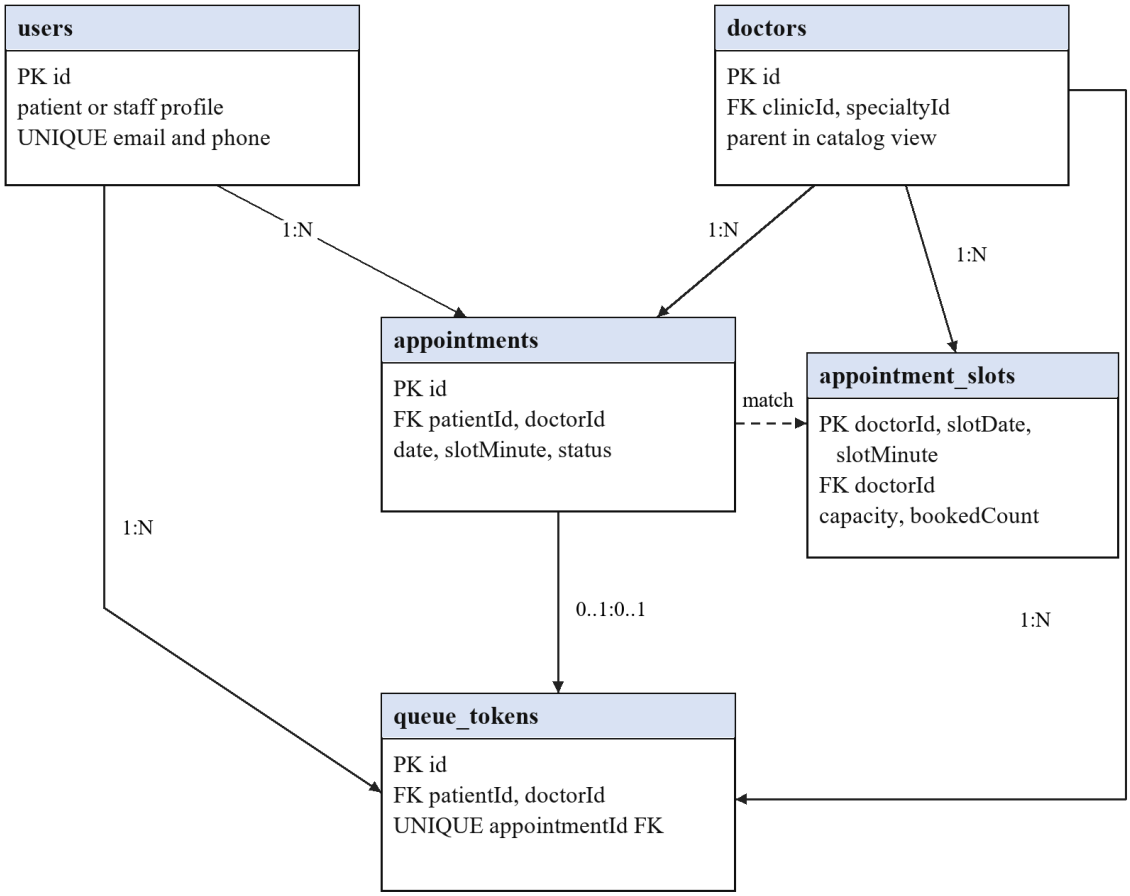


Figure 3.2 Appointment and token relationships

Appointments and tokens reference users and doctors. The patient role is checked by the API; the foreign key itself references users. A nullable unique appointmentId allows a token to be unlinked and prevents several tokens from claiming one appointment. The dashed slot match is maintained by application transactions and triggers; it is not a declared composite foreign key.

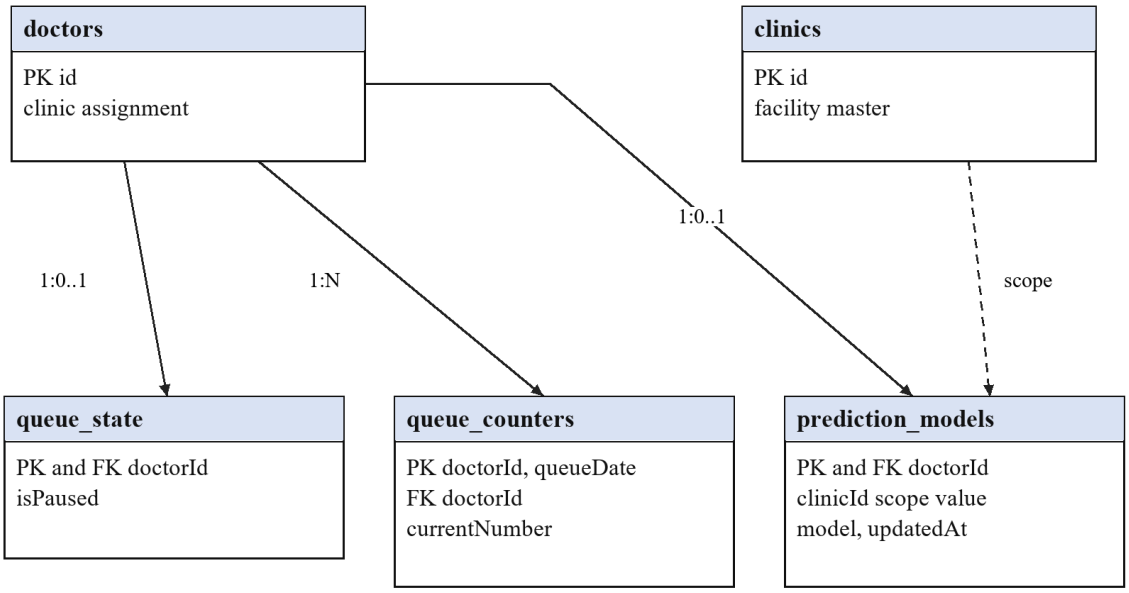


Figure 3.3 Per-doctor state and stored operational models

Counters, pause state and prediction models reference doctors. The dashed clinic-to-model line denotes a scope value without a declared foreign key. Optional staff assignments in users reference a clinic and doctor. Audit events store correlated identifiers without foreign keys; schema_migrations and demo_seed are independent support tables. All relations are listed below.

3.2 Relational Schema

Tables 3.1 to 3.18 list key and selected operational attributes for every base relation. The complete DDL in Appendix A defines all 199 columns, enumerated values, defaults, generated expressions and constraints. PK means primary key, FK means foreign key and UQ means a unique constraint or index.

Table 3.1 clinics

Attribute	Data type	Constraint	Description
id	varchar(100)	PK, NOT NULL	Record identifier
name	varchar(200)	NOT NULL	Display name
district	varchar(100)	NOT NULL	Clinic district
phone	varchar(20)	NOT NULL	Contact number

Table 3.2 specialties

Attribute	Data type	Constraint	Description
id	int	PK, NOT NULL	Record identifier
name	varchar(100)	UQ, NOT NULL	Display name

Table 3.3 languages

Attribute	Data type	Constraint	Description
id	int	PK, NOT NULL	Record identifier
name	varchar(60)	UQ, NOT NULL	Display name

Table 3.4 doctors

Attribute	Data type	Constraint	Description
id	varchar(100)	PK, NOT NULL	Record identifier
clinicId	varchar(100)	NOT NULL, FK clinics	Clinic reference or audit scope
specialtyId	int	NOT NULL, FK specialties	Specialty reference
name	varchar(200)	NOT NULL	Display name
experienceYears	int	NOT NULL	Experience in years, 0 to 70
consultationFee	decimal(10,2)	NOT NULL	Non-negative consultation fee

Table 3.5 doctor_days

Attribute	Data type	Constraint	Description
doctorId	varchar(100)	PK, NOT NULL, FK doctors	Doctor reference
dayOfWeek	ENUM	PK, NOT NULL	Consultation weekday

Table 3.6 doctor_languages

Attribute	Data type	Constraint	Description
doctorId	varchar(100)	PK, NOT NULL, FK doctors	Doctor reference
languageId	int	PK, NOT NULL, FK languages	Language reference

Table 3.7 doctor_interests

Attribute	Data type	Constraint	Description
doctorId	varchar(100)	PK, NOT NULL, FK doctors	Doctor reference
interest	varchar(200)	PK, NOT NULL	Doctor care interest

Table 3.8 users

Attribute	Data type	Constraint	Description
id	varchar(100)	PK, NOT NULL	Record identifier
name	varchar(200)	NOT NULL	Display name
phone	varchar(20)	UQ	Contact number
email	varchar(254)	UQ	Unique contact when present
role	ENUM	NOT NULL	Patient or permitted staff role
clinicId	varchar(100)	FK clinics	Clinic reference or audit scope
doctorId	varchar(100)	FK doctors	Doctor reference

Table 3.9 clinic_settings

Attribute	Data type	Constraint	Description
clinicId	varchar(100)	PK, NOT NULL, FK clinics	Clinic reference or audit scope

Attribute	Data type	Constraint	Description
averageConsultationMinutes	int	NOT NULL	Configured duration baseline
bufferMinutes	int	NOT NULL	Non-negative slot buffer
allowWalkIns	tinyint(1)	NOT NULL	Walk-in configuration

Table 3.10 queue_counters

Attribute	Data type	Constraint	Description
doctorId	varchar(100)	PK, NOT NULL, FK doctors	Doctor reference
queueDate	date	PK, NOT NULL	Clinic-local queue date
currentNumber	int	NOT NULL	Last allocated sequence

Table 3.11 queue_state

Attribute	Data type	Constraint	Description
doctorId	varchar(100)	PK, NOT NULL, FK doctors	Doctor reference
isPaused	tinyint(1)	NOT NULL	Queue pause flag

Table 3.12 appointment_slots

Attribute	Data type	Constraint	Description
doctorId	varchar(100)	PK, NOT NULL, FK doctors	Doctor reference
slotDate	date	PK, NOT NULL	Appointment date
slotMinute	int	PK, NOT NULL	Minutes after midnight
bookedCount	int	NOT NULL	Current reserved places
capacity	int	NOT NULL	Maximum slot places

Table 3.13 appointments

Attribute	Data type	Constraint	Description
id	varchar(100)	PK, NOT NULL	Record identifier
patientId	varchar(100)	NOT NULL, FK users	Patient user reference
doctorId	varchar(100)	NOT NULL, FK doctors	Doctor reference
date	date	NOT NULL	Scheduled appointment date
slotMinute	int	NOT NULL	Minutes after midnight
status	ENUM	NOT NULL	Allowed visit state

Table 3.14 queue_tokens

Attribute	Data type	Constraint	Description
id	varchar(100)	PK, NOT NULL	Record identifier
appointmentId	varchar(100)	UQ, FK appointments	Optional linked appointment
patientId	varchar(100)	NOT NULL, FK users	Patient user reference
doctorId	varchar(100)	NOT NULL, FK doctors	Doctor reference
queueDate	date	NOT NULL	Clinic-local queue date
tokenSeq	int	NOT NULL	Per-doctor daily sequence
status	ENUM	NOT NULL	Allowed visit state
activePatientId	varchar(100)	Generated	Generated active-patient claim

Attribute	Data type	Constraint	Description
servingDoctorId	varchar(100)	Generated	Generated current-consultation claim

Table 3.15 audit_events

Attribute	Data type	Constraint	Description
id	varchar(100)	PK, NOT NULL	Record identifier
clinicId	varchar(100)	Nullable	Clinic reference or audit scope
entityType	varchar(60)	NOT NULL	Audited entity category
entityId	varchar(100)	NOT NULL	Audited record identifier
action	varchar(60)	NOT NULL	Recorded operation
actorId	varchar(100)	NOT NULL	Initiating user or SQL client
requestId	varchar(100)	NOT NULL	Correlation identifier

Table 3.16 prediction_models

Attribute	Data type	Constraint	Description
doctorId	varchar(100)	PK, NOT NULL, FK doctors	Doctor reference
clinicId	varchar(100)	NOT NULL	Clinic reference or audit scope
model	json	NOT NULL	Operational model JSON
updatedAt	datetime(3)	NOT NULL	Last modification timestamp

Table 3.17 schema_migrations

Attribute	Data type	Constraint	Description
version	varchar(30)	PK, NOT NULL	Migration or fixture version
appliedAt	datetime(3)	NOT NULL	Migration application time

Table 3.18 demo_seed

Attribute	Data type	Constraint	Description
version	varchar(30)	PK, NOT NULL	Migration or fixture version
seededAt	datetime(3)	NOT NULL	Fixture installation time

3.3 Schema Definition and Constraints

The schema is created by versioned SQL files at startup. Important rules include unique non-null contact values, non-negative fees, valid schedule bounds, a bookedCount within capacity and foreign keys that reject orphan visits. MySQL uses InnoDB for these relations [1].

```
CREATE TABLE IF NOT EXISTS appointment_slots (
  doctorId VARCHAR(100) NOT NULL, slotDate DATE NOT NULL, slotMinute INT NOT NULL,
  bookedCount INT NOT NULL DEFAULT 0, capacity INT NOT NULL,
  PRIMARY KEY (doctorId, slotDate, slotMinute), FOREIGN KEY (doctorId) REFERENCES doctors(id),
  CHECK (bookedCount >= 0 AND bookedCount <= capacity), CHECK (capacity > 0)
);
```

The generated activePatientId is populated only for waiting/current tokens; a composite unique index with queueDate enforces one active visit per patient per day. Similarly, servingDoctorId is populated only for current tokens. A unique index on servingDoctorId and queueDate permits at most one current consultation per doctor per day. Token numbering is unique on doctorId, queueDate and tokenSeq.

```
UNIQUE active_visit (activePatientId, queueDate),
```

```
UNIQUE one_consultation (servingDoctorId, queueDate),
UNIQUE token_sequence (doctorId, queueDate, tokenSeq)
```

3.4 Normalization

An unnormalized visit sheet could repeat doctor and clinic details beside each patient, while keeping language and weekday lists in single cells. Updating a clinic address or deleting the last visit could then change or remove unrelated master information. The catalog decomposition removes these dependencies.

Normal form	Relations and dependencies	Anomaly addressed
1NF	Atomic master attributes; doctor_days and doctor_languages hold one membership per row.	Repeating weekday and language groups are removed.
2NF	In doctor_languages, the key is (doctorId, languageId). Doctor details stay in doctors; language names stay in languages.	Non-key attributes do not depend on only part of the membership key.
3NF	doctorId determines clinicId and specialtyId; their descriptive attributes are stored in clinics and specialties.	Doctor rows do not repeat clinic address or specialty name.
BCNF assessment	Simple lookup and membership relations satisfy the stated key dependencies. No blanket BCNF claim is made for the operational schema.	Operational copies and derived state are documented rather than hidden.

Representative dependencies are doctorId -> clinicId, specialtyId and doctor attributes; clinicId -> clinic attributes; appointmentId -> patientId, doctorId, date, slotMinute and status; and (doctorId, slotDate, slotMinute) -> capacity and bookedCount. Nullable email and phone values are unique when present, so they are not unconditional candidate keys for every users row.

The implementation deliberately retains operational duplication. An appointment-linked token repeats patient, doctor and date; clinic_settings contains a display copy of the clinic name;

counters and occupancy are derived state. Transactions, triggers and reviewed application paths maintain the tested relationships. Arbitrary raw SQL changes are not equivalent to a validated booking operation.

4 IMPLEMENTATION

SQL source is maintained in backend/sql. The following snippets are taken from the project implementation. The function and daily-flow procedure are used by analytics; triggers execute as part of queue writes. The cursor is a read-only laboratory demonstration. The DBMS is MySQL, so these are MySQL stored programs rather than Oracle PL/SQL.

4.1 Functions

fn_consultation_minutes accepts two DATETIME(3) values and returns a DECIMAL(8,2) duration. It returns NULL for missing, reversed or longer-than-two-hour intervals and returns zero for an identical timestamp. The analytics query calls it when aggregating completed visits. This gives reporting one shared duration rule [2].

```
DELIMITER $$
CREATE FUNCTION fn_consultation_minutes(started DATETIME(3), finished
DATETIME(3))
RETURNS DECIMAL(8,2) DETERMINISTIC NO SQL
BEGIN
  IF started IS NULL OR finished IS NULL OR finished < started OR
TIMESTAMPDIFF(SECOND,started,finished)>7200 THEN RETURN NULL; END IF;
  RETURN ROUND(TIMESTAMPDIFF(MICROSECOND,started,finished)/60000000,2);
END$$
DELIMITER ;
```

The executed positive case returned 12.50 minutes. Missing and reversed inputs returned NULL.

Executed function validation

Actual local MySQL execution | 2026-10-18T20:13:46.247Z

```
SELECT fn_consultation_minutes('2026-10-19 10:00:00','2026-10-19 10:12:30') AS minutes,
fn_consultation_minutes(NULL,'2026-10-19 10:12:30') AS missingInput,
fn_consultation_minutes('2026-10-19 11:00:00','2026-10-19 10:00:00') AS reversedInput;
```

minutes	missingInput	reversedInput
12.50	null	null

Figure 4.1 Executed function results displayed in the SQL verification viewer

4.2 Procedures

sp_daily_patient_flow takes a clinic identifier and an inclusive date range. A recursive date series is left-joined with daily_patient_flow, so a day without matching visits is returned with zero counts instead of disappearing from the chart. The backend calls this procedure for its 14-day analytics series [2].

```
DELIMITER $$
CREATE PROCEDURE sp_daily_patient_flow(IN facility VARCHAR(100), IN from_day
DATE, IN until_day DATE)
READS SQL DATA
BEGIN
  WITH RECURSIVE dates AS (SELECT from_day AS date UNION ALL SELECT
date+INTERVAL 1 DAY FROM dates WHERE date<until_day)
  SELECT dates.date,COALESCE(v.total,0) total,COALESCE(v.completed,0)
completed,COALESCE(v.cancelled,0) cancelled,COALESCE(v.noShow,0) noShow
  FROM dates LEFT JOIN daily_patient_flow v ON v.date=dates.date AND
v.clinicId=facility ORDER BY dates.date;
END$$
DELIMITER ;
```

Executed daily flow procedure

Actual local MySQL execution | 2026-10-18T20:13:46.247Z

```
CALL sp_daily_patient_flow('clinic-1','2026-10-06','2026-10-19');
```

date	total	completed	cancelled	noShow
2026-10-06	43	37	2	4
2026-10-07	40	36	1	3
2026-10-08	37	34	0	3
2026-10-09	39	37	0	2
2026-10-10	41	40	0	1
2026-10-11	43	42	0	1
2026-10-12	40	40	0	0
2026-10-13	48	16	0	0
2026-10-14	8	0	0	0
2026-10-15	8	0	0	0
2026-10-16	8	0	0	0
2026-10-17	0	0	0	0
2026-10-18	0	0	0	0
2026-10-19	0	0	0	0

Figure 4.2 Recorded daily-flow procedure output including zero-volume days

4.3 Packages

MySQL 8.4 does not implement Oracle-style package specifications and bodies. Consequently, this project does not claim a PL/SQL package. The report retains this template section to make the platform choice explicit. Related SQL routines are deployed together in 002_routines.sql, while connection and repository functions are grouped in TypeScript modules; those modules are not database packages.

```
SELECT ROUTINE_NAME, ROUTINE_TYPE
FROM information_schema.ROUTINES
WHERE ROUTINE_SCHEMA = DATABASE();
```

The recorded database contains one stored function and two stored procedures. Package syntax from the Oracle examples in the template has not been inserted into the MySQL project.

4.4 Triggers

All three triggers operate at row level on queue_tokens. validate_token_transition runs BEFORE UPDATE and rejects invalid state changes or calling a patient while the doctor queue is paused. audit_token_insert runs AFTER INSERT and creates an audit event. synchronize_token_update runs AFTER UPDATE, maps token state to appointment state, releases cancelled/skipped slot occupancy and records the transition [3].

```
DELIMITER $$
CREATE TRIGGER validate_token_transition BEFORE UPDATE ON queue_tokens FOR
EACH ROW
BEGIN
  IF NEW.status<>OLD.status AND NOT (
    OLD.status='waiting' AND NEW.status IN ('current','skipped','cancelled') OR
    OLD.status='current' AND NEW.status IN ('completed','skipped')
  ) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='Invalid queue status
transition'; END IF;
  IF NEW.status='current' AND EXISTS(SELECT 1 FROM queue_state WHERE
doctorId=NEW.doctorId AND isPaused=TRUE)
    THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='Resume the queue before
calling a patient'; END IF;
END$$
DELIMITER ;
```

```
DELIMITER $$
CREATE TRIGGER synchronize_token_update AFTER UPDATE ON queue_tokens FOR
EACH ROW
BEGIN
  IF NEW.status<>OLD.status THEN
    UPDATE appointments SET status=CASE NEW.status WHEN 'current' THEN
'in_consultation'
      WHEN 'skipped' THEN 'no_show' ELSE NEW.status END,
      version=version+1,updatedAt=UTC_TIMESTAMP(3)
    WHERE id=NEW.appointmentId;
    IF NEW.status IN ('cancelled','skipped') THEN
```

```
    UPDATE appointment_slots s JOIN appointments a ON a.doctorId=s.doctorId AND
a.date=s.slotDate AND a.slotMinute=s.slotMinute
    SET s.bookedCount=GREATEST(0,s.bookedCount-1) WHERE
a.id=NEW.appointmentId;
  END IF;
  INSERT INTO
audit_events(id,clinicId,entityType,entityId,action,actorId,requestId,details)
  SELECT UUID(),clinicId,'token',NEW.id,'status_changed',COALESCE(@qf_actor,'sql-
client'),COALESCE(@qf_request,UUID()),
    JSON_OBJECT('from',OLD.status,'to',NEW.status,'queueDate',NEW.queueDate) FROM
doctors WHERE id=NEW.doctorId;
 END IF;
END$$
DELIMITER ;
```

The trigger demonstration uses disposable synthetic records inside one transaction. Before cancellation the appointment was booked, the token waiting and bookedCount equal to one. After the token update, both records were cancelled and bookedCount was zero. The final ROLLBACK left zero demonstration users, appointments, tokens and audit events. The browser workflow independently verified the same linked-state behaviour on token A-001.

4.5 Cursors

sp_doctor_workload_cursor declares an explicit cursor over the clinic roster. It opens the cursor, fetches one doctor at a time, counts waiting tokens for the requested date and appends a JSON result. A NOT FOUND handler terminates the loop, and the cursor is closed before the result is returned. It demonstrates procedural row traversal; normal operational summaries use set-based aggregation [4].

```
DELIMITER $$
CREATE PROCEDURE sp_doctor_workload_cursor(IN facility VARCHAR(100), IN
visit_day DATE)
READS SQL DATA
BEGIN
 DECLARE finished BOOLEAN DEFAULT FALSE;
 DECLARE doctor_key VARCHAR(100);
```

```
  DECLARE doctor_name VARCHAR(200);
  DECLARE waiting_count INT;
  DECLARE result JSON DEFAULT JSON_ARRAY();
  DECLARE roster CURSOR FOR SELECT id,name FROM doctors WHERE
clinicId=facility ORDER BY name;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET finished=TRUE;
  OPEN roster;
  read_roster: LOOP
    FETCH roster INTO doctor_key,doctor_name;
    IF finished THEN LEAVE read_roster; END IF;
    SELECT COUNT(*) INTO waiting_count FROM queue_tokens WHERE
doctorId=doctor_key AND queueDate=visit_day AND status='waiting';
    SET
result=JSON_ARRAY_APPEND(result,'$',JSON_OBJECT('doctorId',doctor_key,'name',doc
tor_name,'waiting',waiting_count));
  END LOOP;
  CLOSE roster;
  SELECT result AS workload;
END$$
DELIMITER ;
```

Executed roster cursor

Actual local MySQL execution | 2026-10-18T20:13:46.247Z

```
CALL sp_doctor_workload_cursor('clinic-1','2026-10-13');
```

name	waiting	doctorId
Dr. Aditya Mukherjee	3	doc-13
Dr. Amitav Bhattacharya	3	doc-10
Dr. Ananya Sharma	3	doc-1
Dr. Debabrata Roy	3	doc-4
Dr. Rahul Sen	3	doc-2
Dr. Sana Das	3	doc-12
Dr. Sunita Banerjee	3	doc-3
Dr. Vihaan Das	3	doc-11

Figure 4.3 Executed cursor result formatted from its returned JSON

4.6 SQL Search and Transaction Control

Directory search is performed by MySQL, including specialty and language membership, clinic scope, ordering and LIMIT/OFFSET. Values are bound as parameters. Sort expressions are selected from a whitelist rather than concatenated from free-form user input.

```
SELECT d.id, d.name, s.name AS specialty,
    c.name AS clinic, d.consultationFee
FROM doctors d
JOIN specialties s ON s.id = d.specialtyId
JOIN clinics c ON c.id = d.clinicId
WHERE s.name = 'Gynecology' AND d.status = 'active'
ORDER BY d.consultationFee, d.name;
```

Executed specialty search

Actual local MySQL execution | 2026-10-18T20:13:46.247Z

```
SELECT d.name,s.name AS specialty,c.name AS clinic,d.consultationFee FROM doctors d JOIN
specialties s ON s.id=d.specialtyId JOIN clinics c ON c.id=d.clinicId WHERE
s.name='Gynecology' AND d.status='active' ORDER BY d.consultationFee,d.name;
```

name	specialty	clinic	consultationFee
Dr. Meera Patel	Gynecology	Willow Community Care	350.00
Dr. Aditi Roy	Gynecology	Riverstone Medical Centre	550.00
Dr. Sourav Ganguly	Gynecology	Riverstone Medical Centre	750.00
Dr. Aditi Chatterjee	Gynecology	Maple Outpatient Centre	850.00

Figure 4.4 Joined specialty query and its actual sorted result

Booking uses a pooled connection with BEGIN, COMMIT and ROLLBACK. The transaction locks the doctor and slot rows, checks capacity, inserts linked appointment/token records and updates occupancy. Queue calls and pause operations use a consistent doctor-first lock order. This avoids implementing concurrency protection solely as a JavaScript pre-check [5].

```
SELECT status FROM doctors WHERE id = ? FOR UPDATE;
SELECT bookedCount, capacity
FROM appointment_slots
WHERE doctorId = ? AND slotDate = ? AND slotMinute = ?
FOR UPDATE;
```

5 FRONT END AND ANDROID INTEGRATION

5.1 User Interface

Patient screens expose the directory, available slots, booking confirmation, appointment history and queue information. Staff screens expose clinic-scoped doctors, appointments, queues and analytics. The following images are actual screenshots of the running local web application, captured after API responses completed.

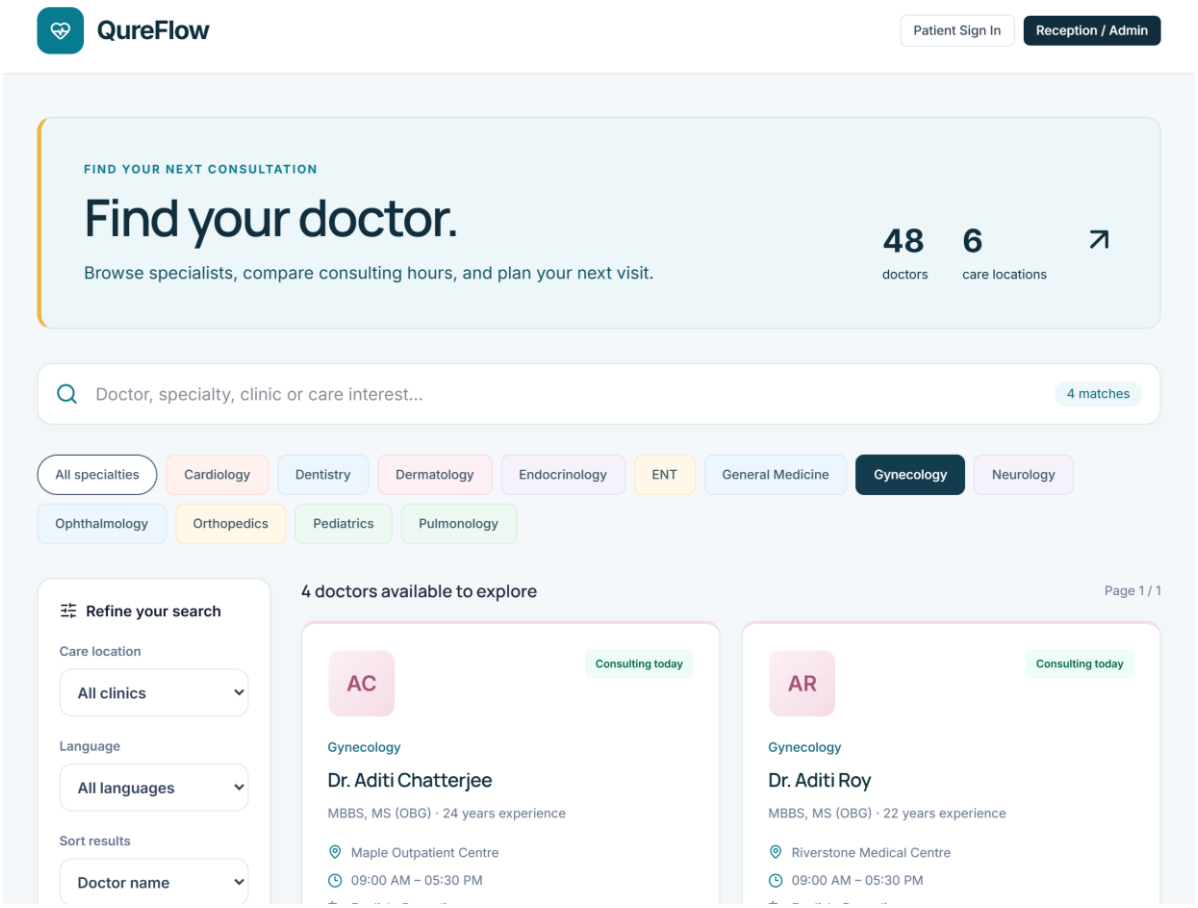


Figure 5.1 Web directory filtered to Gynecology using database results

The specialty selection is sent to the SQL search endpoint. The displayed doctors come from joined clinic and specialty records. Filtering and the selected sort order are applied by MySQL, with paginated results returned to the browser.

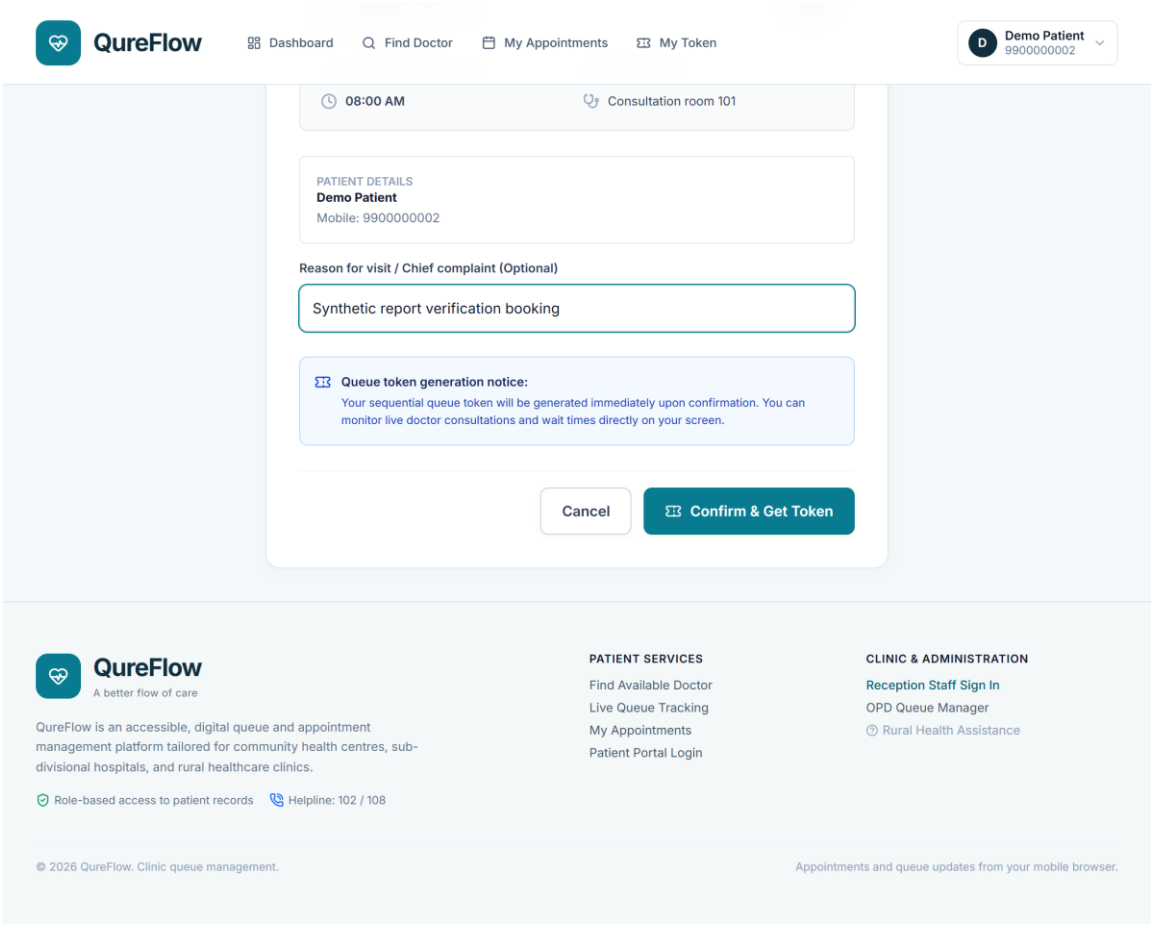


Figure 5.2 Patient confirmation before submitting the verification booking

The confirmation page shows the selected doctor, date and slot before any write is submitted. For this execution, the signed-in synthetic patient selected an 08:00 appointment on 20 October. The browser then submitted one booking request; the returned identifier was used for the independent SQL check.

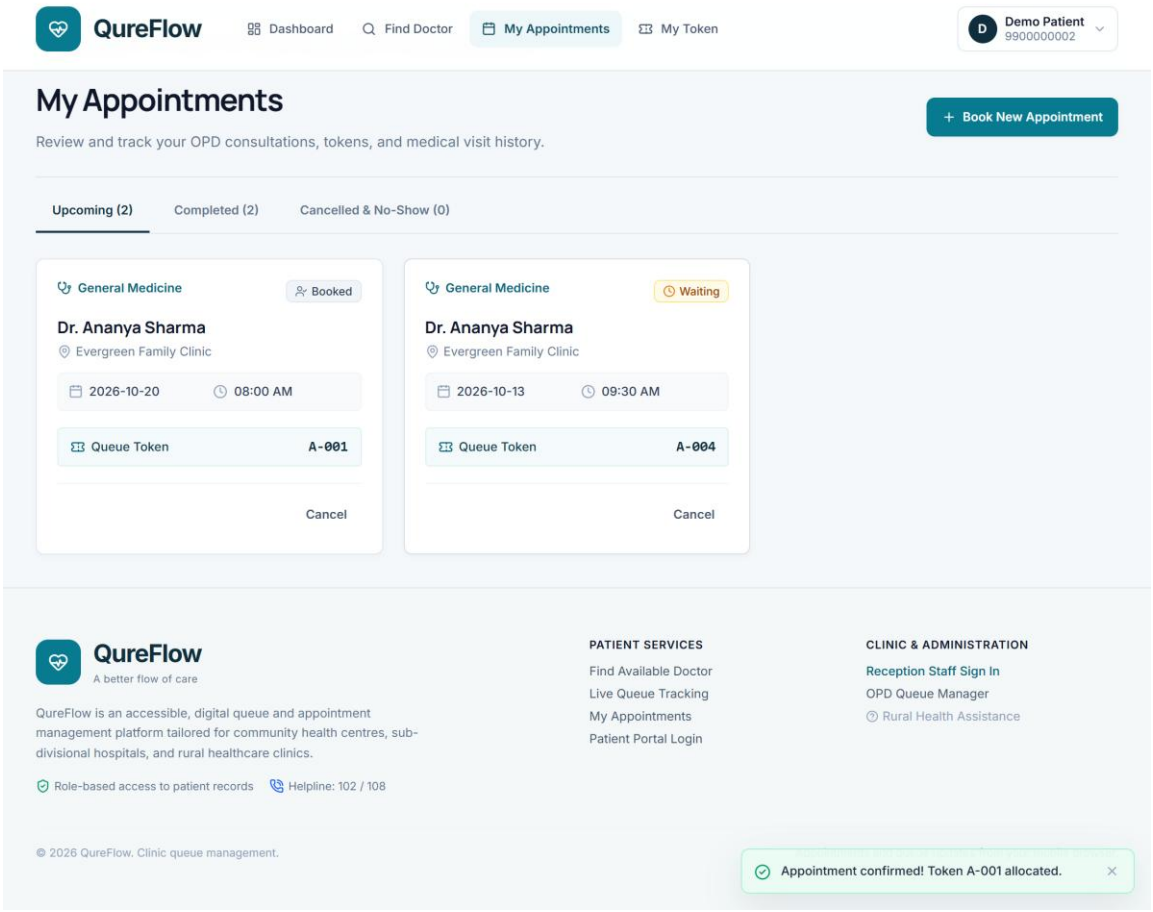


Figure 5.3 Confirmed browser booking for 20 October 2026

The appointment history displays the returned token A-001. At this point SQL showed an appointment in booked state, a linked waiting token and one reserved place. The comparison in Section 6 records the exact before and after values.

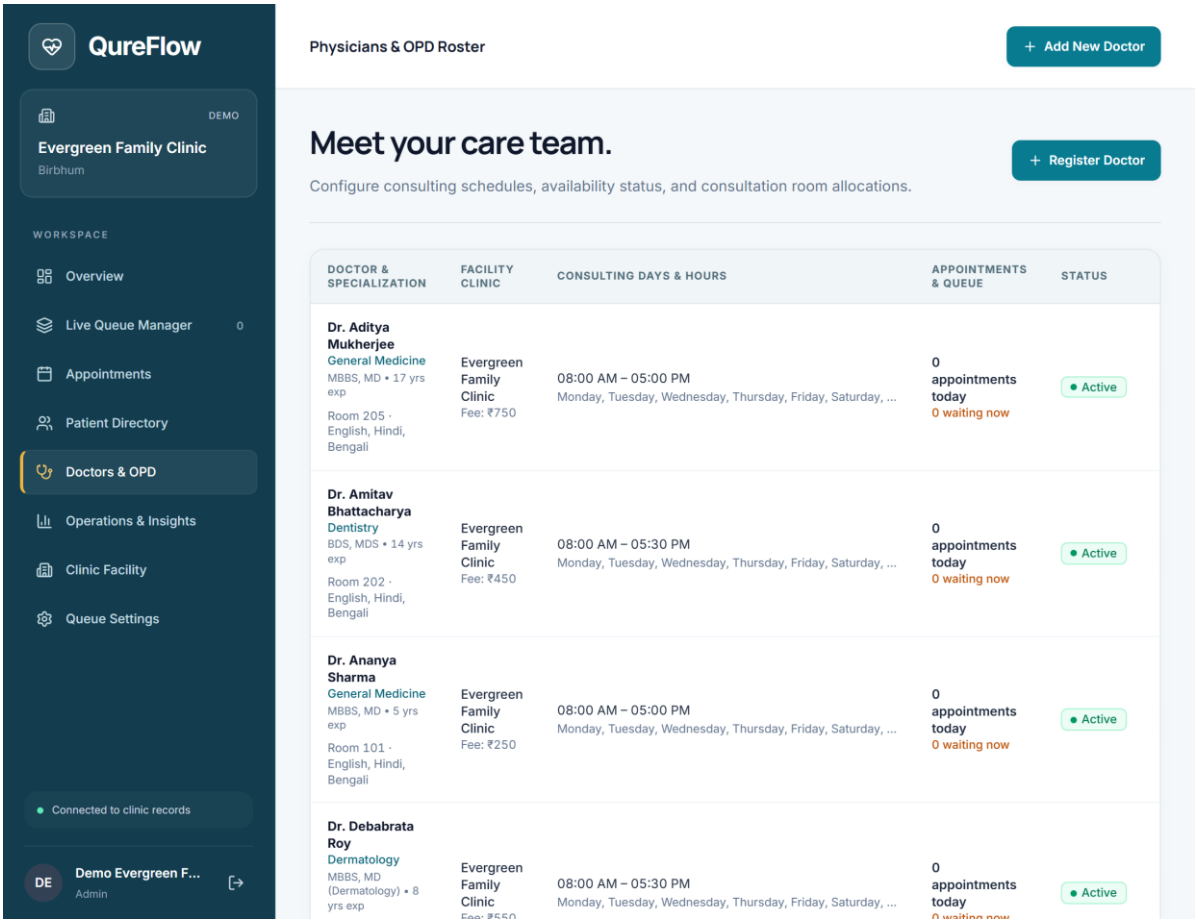


Figure 5.4 Clinic-scoped doctor administration after staff sign-in

Staff sign-in exposes the assigned clinic workflow. SQL queries and API authorization jointly apply clinic scope. Cross-clinic modification and patient attempts to use staff-only appointment filters were rejected in the recorded regression cases.

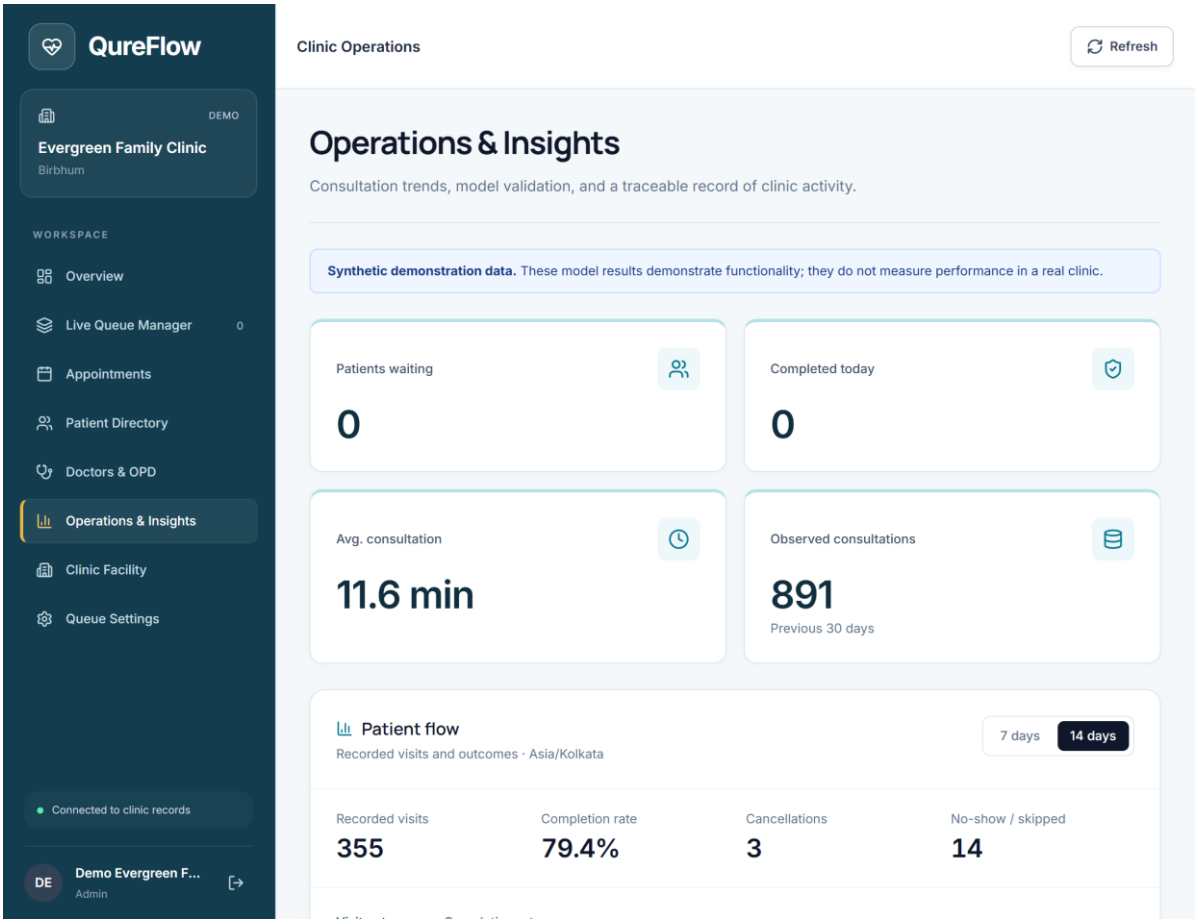


Figure 5.5 Clinic analytics summary computed from recorded SQL data

5.2 Integration with the DBMS

The React client sends HTTP requests to Express. The API validates the session and role, applies clinic scope and calls the MySQL repository through mysql2/promise. Database triggers run within the same transaction as the write that caused them. The API returns success only after the database operation has completed. Database passwords remain in the backend configuration.

```
const [rows] = await connection.execute(
 'SELECT version FROM users WHERE id=? FOR UPDATE',
 [userId]
);
```

The connection pool is configured for the local MySQL endpoint, uses UTC timestamps and disables multiple SQL statements per request. On a failed transaction the connection rolls back, resets the audit context variables and returns to the pool. The database stores clinic-local dates separately from event timestamps.

5.3 Android Application

Capacitor packages the React interface as com.qureflow.dbms. The verified debug APK is version 1.3, code 4, targeting SDK 36 with minimum API 24. It calls the same API rather than connecting directly to MySQL [6].

Connection discovery combines mDNS with a bounded UDP fallback for Wi-Fi/hotspot interfaces. Candidate addresses receive anonymous health checks before a saved clinic is selected. Reconnection pauses requests and cancels in-flight calls; interrupted writes are not replayed. Doctor reads reload after verification without resetting filters.

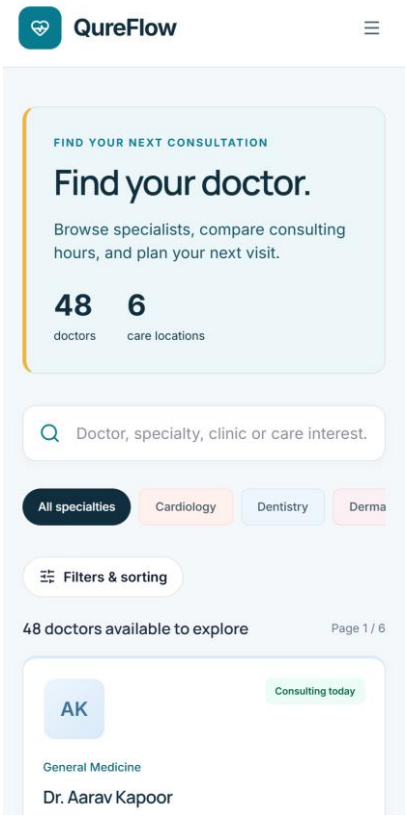


Figure 5.6 Responsive browser view at phone dimensions; this is not a native APK screenshot

APK compilation, package contents, signing and the absence of SQL credentials were verified. Hotspot discovery failed in the earlier build; final acceptance of the new Android hotspot path remains pending. A responsive browser image does not replace a native device execution test [7].

6 VERIFICATION AND TESTING

6.1 Test Cases

The MySQL suite creates a UUID-named test database, loads isolated synthetic records, executes the API/database assertions and removes the test database. It does not run these destructive test cases against the normal demonstration schema. The fresh run completed 24 individual cases with no failures; Node also reports the parent suite as one passing test entry.

Table 6.1 Executed database cases 1 to 8

TC ID	Description	Input	Expected output	Actual output	Result
TC01	Joined directory	Specialty, Hindi and experience order	Correct filtering and order	Matched expected rows	Pass
TC02	SQL injection	Injected search and sort expression	No injected results; invalid sort rejected	Search empty; HTTP 400	Pass
TC03	Pagination	Unknown specialty and page 999	Empty page with accurate total	Totals preserved	Pass
TC04	Authorization	Anonymous read and cross-clinic edits	401 or 403 rejection	Requests denied	Pass
TC05	Duplicate visit	Two concurrent joins for one patient	One active visit only	One succeeded	Pass
TC06	Foreign key	Missing patient reference	Reject orphan token	FK error observed	Pass
TC07	Priority order	Normal and emergency entries	Explicit emergency first	Correct next token	Pass

TC ID	Description	Input	Expected output	Actual output	Result
TC08	Consultation claim	Second current token for doctor/date	Unique constraint rejects second	One current token	Pass

Table 6.2 Executed database cases 9 to 16

TC ID	Description	Input	Expected output	Actual output	Result
TC09	Patient privacy	Another patient queue/cancel request	Conceal identity and deny mutation	Identity hidden; 403	Pass
TC10	Trigger transition	Complete then reopen token	Audit and invalid-state rejection	Reopening rejected	Pass
TC11	Pause and resume	Call paused queue, then resume	Reject while paused; succeed later	Both assertions passed	Pass
TC12	Atomic booking	Book available future slot	Linked appointment/token; occupancy 1	Linked rows verified	Pass
TC13	Full capacity	Book after capacity reached	Conflict without orphan records	Transaction rolled back	Pass
TC14	Cancellation	Cancel linked visit	Both cancelled; occupancy released	Both states; count 0	Pass
TC15	Profile concurrency	Stale version and duplicate email	Reject conflicting updates	Both rejected	Pass
TC16	Identity protection	New ID with existing identity	Existing account not overwritten	Original unchanged	Pass

Table 6.3 Executed database cases 17 to 24

TC ID	Description	Input	Expected output	Actual output	Result
TC17	Staff pagination	Clinic-scoped appointment page	Correct page; patient forbidden	Rows and 403 verified	Pass
TC18	Stored objects	Function, procedure, views, cursor	12.5 minutes; 14 days; roster	Expected outputs matched	Pass
TC19	Scoped filters	Injected patient filter and clinic read	No escape from authorized scope	Empty/authorized rows	Pass
TC20	Buffer capacity	Change buffer and reuse slot	Newly opened place can be booked	Booking succeeded	Pass
TC21	CHECK constraints	Negative fee and excess occupancy	Reject invalid rows	CHECK errors observed	Pass
TC22	Invalid timing	Missing, reversed and long intervals	NULL results; zero for equal times	Expected values matched	Pass
TC23	Concurrent capacity	Three patients, one remaining place	Exactly one successful booking	One success; count 1	Pass
TC24	API persistence	HTTP booking then cancellation	SQL linked rows and both states	201 then 200; SQL matched	Pass

6.2 Output Screenshots

MySQL test execution output

24 individual cases plus the parent suite

```
  ✓ CHECK constraints reject negative fees and impossible slot occupancy
(6.5688ms)
  ✓ function returns NULL for missing, reversed and implausible timings
(4.0115ms)
  ✓ concurrent bookings from different patients cannot overfill the last slot
(89.0899ms)
  ✓ a booking submitted through the HTTP API is visible in SQL and cancels both
records (92.7596ms)
✓ MySQL DBMS integration (4671.3965ms)
ℹ tests 25
ℹ suites 0
ℹ pass 25
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 4898.2027
```

Figure 6.1 Actual test-run summary showing zero failures

Browser booking verified in SQL

Actual local MySQL execution | 2026-10-18T20:13:46.247Z

```
Token A-001 | Appointment apt-7fb41770-fa6a-4da3-a025-33261330f765
SELECT a.id,a.date,a.status AS appointmentStatus,t.status AS tokenStatus,s.bookedCount FROM
appointments a JOIN queue_tokens t ON t.appointmentId=a.id JOIN appointment_slots s ON
s.doctorId=a.doctorId AND s.slotDate=a.date AND s.slotMinute=a.slotMinute WHERE a.id='apt-
7fb41770-fa6a-4da3-a025-33261330f765'
```

phase	date	appointment	token	bookedCount
After booking	2026-10-20	booked	waiting	1
After cancellation	2026-10-20	cancelled	cancelled	0

Figure 6.2 SQL records captured before and after browser cancellation

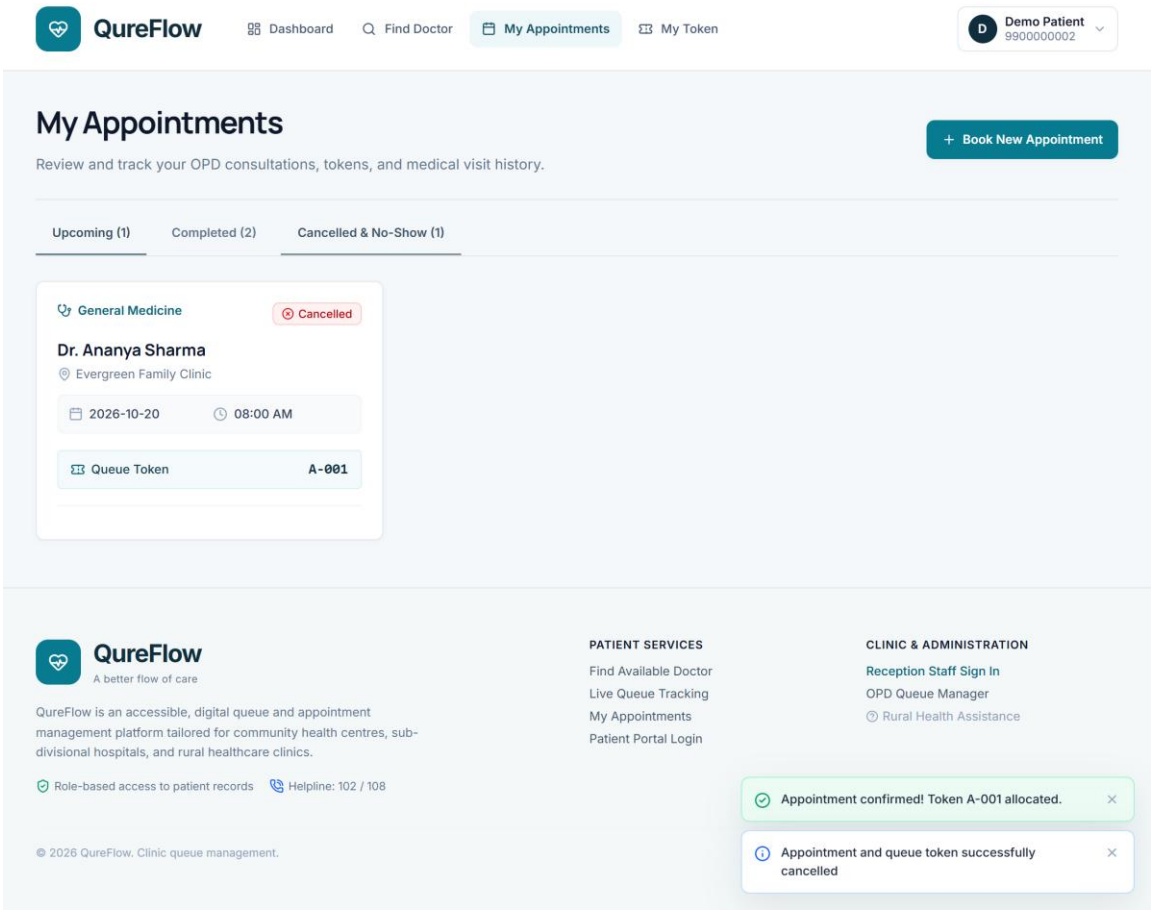


Figure 6.3 Browser appointment history after the verification cancellation

6.3 Validation of Functionality

The browser confirmation produced appointment apt-7fb41770-fa6a-4da3-a025-33261330f765 and token A-001. The independent SQL read found appointmentStatus=booked, tokenStatus=waiting and bookedCount=1. Cancellation through the browser changed both statuses to cancelled and released the slot to bookedCount=0. The final database contains 6,196 appointments and 6,196 queue tokens, including this cancelled verification record.

The SQL function returned 12.50 minutes; invalid timing inputs produced NULL. The daily procedure returned all 14 requested dates, including zero-volume days. The cursor returned the eight doctors assigned to clinic-1 for the selected fixture date. Separate negative tests verified foreign keys, CHECK constraints, illegal transitions, identity conflicts and capacity races.

Additional recorded checks passed 18 mobile/client tests and five discovery tests. These validate reconnection logic, protocol behaviour and local discovery exchange. They do not prove delivery of multicast or broadcast packets on every Android hotspot. APK package verification is kept separate from device acceptance.

7 RESULTS AND DISCUSSION

7.1 Achievements

Recorded item	Result
Database model	18 base tables, 199 columns, 18 declared foreign keys and four views
Demonstration scale	Six fictional clinics, 48 doctors and 737 patient profiles
Visit records	6,196 appointments and 6,196 tokens after browser verification
Stored objects	One function, two procedures and three triggers; one procedure uses a cursor
SQL verification	All 24 individual MySQL integration cases passed
Web integration	Browser booking and cancellation matched independent SQL reads
Android packaging	Version 1.3 built and signed; device hotspot acceptance pending

The evidence demonstrates relational consistency, SQL filtering and rejection of conflicting writes. It supports a reproducible local workflow, without establishing a real-world waiting-time reduction or production certification.

7.2 Challenges Faced

Migrating the workflow required explicit relations and SQL queries in place of document-oriented access patterns. Concurrent booking needed row locks and conditional constraints; cancellation needed one consistent update to the token, appointment and capacity counter. Failed operations were verified to roll back.

The phone could reach the API while hotspot discovery failed. A direct discovery fallback was added and locally tested. A cancelled doctor search also needed to reload after reconnection; a component regression verified the read refresh.

7.3 Possible Improvements

Next steps are native hotspot acceptance, HTTPS enrollment, backup and load tests, restricted database privileges and reconciliation of direct edits.

8 CONCLUSION

8.1 Summary of Learning Outcomes

QureFlow applies relational design to a queue-management workflow. The project combines normalized master data with explicit operational state, uses SQL joins and grouping for useful application queries, and protects writes with transactions, locks, foreign keys, checks, unique indexes and triggers. It also distinguishes a reusable stored function, a set-based reporting procedure and an explicit cursor procedure.

The most important outcome is reproducible verification. Positive and negative database cases, a real browser booking/cancellation and the matching SQL output demonstrate how interface actions affect stored records. The work also shows why API authorization, schema rules and network verification address different parts of the application rather than replacing one another.

8.2 Future Scope

The local demonstration can be extended toward clinic deployment after device, operational and security acceptance. Waiting-time models should be evaluated on consented operational timings and used as transparent scheduling guidance. They should not infer diagnoses or perform clinical triage. The original AWS variant and this MySQL laboratory variant should continue to be maintained as clearly separated configurations.

9 REFERENCES

[1] Oracle, "FOREIGN KEY Constraints," MySQL 8.4 Reference Manual. [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/create-table-foreign-keys.html. Accessed: Oct. 19, 2026.

[2] Oracle, "CREATE PROCEDURE and CREATE FUNCTION Statements," MySQL 8.4 Reference Manual. [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/create-procedure.html. Accessed: Oct. 19, 2026.

[3] Oracle, "CREATE TRIGGER Statement," MySQL 8.4 Reference Manual. [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/create-trigger.html. Accessed: Oct. 19, 2026.

[4] Oracle, "Cursors," MySQL 8.4 Reference Manual. [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/cursors.html. Accessed: Oct. 19, 2026.

[5] Oracle, "Locking Reads," MySQL 8.4 Reference Manual. [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html. Accessed: Oct. 19, 2026.

[6] Capacitor, "Custom Native Android Code," Capacitor Documentation, v8. [Online]. Available: https://capacitorjs.com/docs/android/custom-code. Accessed: Oct. 19, 2026.

[7] Google, "Use network service discovery," Android Developers. [Online]. Available: https://developer.android.com/develop/connectivity/wifi/use-nsd. Accessed: Oct. 19, 2026.

[8] K. Gupta and S. Sinha, "QureFlow MySQL DBMS variant," project source and local execution records, 2026. Internal files: backend/sql/001_schema.sql, 002_routines.sql, backend/src/repositories/mysqlRepo.ts and backend/tests/mysql.test.ts.

10 APPENDICES

Appendix A Complete Schema Definition

The following is the executed project schema source. It defines all base relations and the four views; the report dictionary in Section 3.2 lists selected attributes for readability.

```
-- QureFlow DBMS variant. MySQL 8.4 / InnoDB; all data is local and synthetic.
CREATE TABLE IF NOT EXISTS schema_migrations (
  version VARCHAR(30) PRIMARY KEY, appliedAt DATETIME(3) NOT NULL
DEFAULT CURRENT_TIMESTAMP(3)
);
CREATE TABLE IF NOT EXISTS clinics (
  id VARCHAR(100) PRIMARY KEY, name VARCHAR(200) NOT NULL, address
VARCHAR(500) NOT NULL,
  district VARCHAR(100) NOT NULL, state VARCHAR(100) NOT NULL, phone
VARCHAR(20) NOT NULL,
  email VARCHAR(254) NOT NULL, openingHours VARCHAR(100) NOT NULL,
emergencyContact VARCHAR(30) NOT NULL,
  metadata JSON, updatedAt DATETIME(3) NOT NULL DEFAULT
CURRENT_TIMESTAMP(3)
);
CREATE TABLE IF NOT EXISTS specialties (
  id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100) NOT NULL
UNIQUE
);
CREATE TABLE IF NOT EXISTS languages (
  id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(60) NOT NULL
UNIQUE
);
CREATE TABLE IF NOT EXISTS doctors (
  id VARCHAR(100) PRIMARY KEY, clinicId VARCHAR(100) NOT NULL, specialtyId
INT NOT NULL,
  name VARCHAR(200) NOT NULL, qualifications VARCHAR(300) NOT NULL,
  experienceYears INT NOT NULL CHECK (experienceYears BETWEEN 0 AND 70),
```

```
  consultationFee DECIMAL(10,2) NOT NULL CHECK (consultationFee >= 0),
  startMinute INT NOT NULL, endMinute INT NOT NULL, slotDurationMinutes INT NOT
NULL CHECK (slotDurationMinutes BETWEEN 2 AND 30),
  status ENUM('active','inactive') NOT NULL DEFAULT 'active', avatar VARCHAR(1000),
  bio TEXT, roomNumber VARCHAR(30), department VARCHAR(100), registrationId
VARCHAR(100), metadata JSON,
  FOREIGN KEY (clinicId) REFERENCES clinics(id), FOREIGN KEY (specialtyId)
REFERENCES specialties(id),
  CHECK (startMinute >= 0 AND endMinute <= 1440 AND endMinute > startMinute),
  INDEX doctors_directory (specialtyId, clinicId, status), INDEX doctors_fee
(consultationFee), INDEX doctors_experience (experienceYears)
);
CREATE TABLE IF NOT EXISTS doctor_days (
  doctorId VARCHAR(100) NOT NULL, dayOfWeek
ENUM('Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday') NOT NULL,
  PRIMARY KEY (doctorId, dayOfWeek), FOREIGN KEY (doctorId) REFERENCES
doctors(id)
);
CREATE TABLE IF NOT EXISTS doctor_languages (
  doctorId VARCHAR(100) NOT NULL, languageId INT NOT NULL,
  PRIMARY KEY (doctorId, languageId), FOREIGN KEY (doctorId) REFERENCES
doctors(id), FOREIGN KEY (languageId) REFERENCES languages(id)
);
CREATE TABLE IF NOT EXISTS doctor_interests (
  doctorId VARCHAR(100) NOT NULL, interest VARCHAR(200) NOT NULL,
  PRIMARY KEY (doctorId, interest), FOREIGN KEY (doctorId) REFERENCES
doctors(id)
);
CREATE TABLE IF NOT EXISTS users (
  id VARCHAR(100) PRIMARY KEY, name VARCHAR(200) NOT NULL, phone
VARCHAR(20) UNIQUE, email VARCHAR(254) UNIQUE,
  role ENUM('patient','doctor','health_worker','admin') NOT NULL,
  dob DATE, gender ENUM('Male','Female','Other'), staffId VARCHAR(100) UNIQUE,
doctorId VARCHAR(100),
```

```
 clinicId VARCHAR(100), passwordHash VARCHAR(255), cognitoSub VARCHAR(200)
UNIQUE, sessionVersion INT NOT NULL DEFAULT 0,
 version INT NOT NULL DEFAULT 1, metadata JSON, createdAt DATETIME(3) NOT
NULL, updatedAt DATETIME(3) NOT NULL,
 FOREIGN KEY (clinicId) REFERENCES clinics(id), FOREIGN KEY (doctorId)
REFERENCES doctors(id)
);
CREATE TABLE IF NOT EXISTS clinic_settings (
 clinicId VARCHAR(100) PRIMARY KEY, averageConsultationMinutes INT NOT NULL
CHECK (averageConsultationMinutes BETWEEN 1 AND 120),
 bufferMinutes INT NOT NULL DEFAULT 2 CHECK (bufferMinutes >= 0), allowWalkIns
BOOLEAN NOT NULL DEFAULT TRUE,
 allowEmergencyPriority BOOLEAN NOT NULL DEFAULT TRUE, notifyOnTwoAhead
BOOLEAN NOT NULL DEFAULT FALSE,
 autoTokenPrefixByDoctor BOOLEAN NOT NULL DEFAULT TRUE, clinicName
VARCHAR(200) NOT NULL, contactNumber VARCHAR(30) NOT NULL,
 FOREIGN KEY (clinicId) REFERENCES clinics(id)
);
CREATE TABLE IF NOT EXISTS queue_counters (
 doctorId VARCHAR(100) NOT NULL, queueDate DATE NOT NULL, currentNumber
INT NOT NULL DEFAULT 0,
 PRIMARY KEY (doctorId, queueDate), FOREIGN KEY (doctorId) REFERENCES
doctors(id)
);
CREATE TABLE IF NOT EXISTS queue_state (
 doctorId VARCHAR(100) PRIMARY KEY, isPaused BOOLEAN NOT NULL DEFAULT
FALSE,
 FOREIGN KEY (doctorId) REFERENCES doctors(id)
);
CREATE TABLE IF NOT EXISTS appointment_slots (
 doctorId VARCHAR(100) NOT NULL, slotDate DATE NOT NULL, slotMinute INT NOT
NULL,
 bookedCount INT NOT NULL DEFAULT 0, capacity INT NOT NULL,
```

```
  PRIMARY KEY (doctorId, slotDate, slotMinute), FOREIGN KEY (doctorId)
REFERENCES doctors(id),
  CHECK (bookedCount >= 0 AND bookedCount <= capacity), CHECK (capacity > 0)
);
CREATE TABLE IF NOT EXISTS appointments (
  id VARCHAR(100) PRIMARY KEY, patientId VARCHAR(100) NOT NULL, doctorId
VARCHAR(100) NOT NULL,
  date DATE NOT NULL, slotMinute INT NOT NULL, tokenNumber VARCHAR(100)
NOT NULL,
  status ENUM('booked','waiting','in_consultation','completed','cancelled','no_show') NOT
NULL,
  priority ENUM('normal','emergency') NOT NULL, isWalkIn BOOLEAN NOT NULL
DEFAULT FALSE,
  notes TEXT, version INT NOT NULL DEFAULT 1, metadata JSON, createdAt
DATETIME(3) NOT NULL, updatedAt DATETIME(3) NOT NULL,
  FOREIGN KEY (patientId) REFERENCES users(id), FOREIGN KEY (doctorId)
REFERENCES doctors(id),
  INDEX appointments_patient (patientId, date), INDEX appointments_doctor (doctorId,
date, slotMinute)
);
CREATE TABLE IF NOT EXISTS queue_tokens (
  id VARCHAR(100) PRIMARY KEY, appointmentId VARCHAR(100) UNIQUE,
  patientId VARCHAR(100) NOT NULL, doctorId VARCHAR(100) NOT NULL,
queueDate DATE NOT NULL,
  tokenSeq INT NOT NULL, tokenNumber VARCHAR(100) NOT NULL, scheduledMinute
INT,
  priority ENUM('normal','emergency') NOT NULL DEFAULT 'normal', isWalkIn
BOOLEAN NOT NULL DEFAULT FALSE,
  status ENUM('waiting','current','completed','skipped','cancelled') NOT NULL,
  createdAt DATETIME(3) NOT NULL, calledAt DATETIME(3), servedAt DATETIME(3),
  version INT NOT NULL DEFAULT 1, updatedAt DATETIME(3) NOT NULL, metadata
JSON,
  activePatientId VARCHAR(100) GENERATED ALWAYS AS (CASE WHEN status IN
('waiting','current') THEN patientId ELSE NULL END) STORED,
```

```
  servingDoctorId VARCHAR(100) GENERATED ALWAYS AS (CASE WHEN status =
'current' THEN doctorId ELSE NULL END) STORED,
  UNIQUE active_visit (activePatientId, queueDate), UNIQUE one_consultation
(servingDoctorId, queueDate),
  UNIQUE token_sequence (doctorId, queueDate, tokenSeq),
  FOREIGN KEY (patientId) REFERENCES users(id), FOREIGN KEY (doctorId)
REFERENCES doctors(id),
  FOREIGN KEY (appointmentId) REFERENCES appointments(id),
  CHECK (servedAt IS NULL OR calledAt IS NOT NULL AND servedAt >= calledAt),
  INDEX queue_waiting (doctorId, queueDate, status, priority, tokenSeq)
);
CREATE TABLE IF NOT EXISTS audit_events (
  id VARCHAR(100) PRIMARY KEY, clinicId VARCHAR(100), entityType
VARCHAR(60) NOT NULL, entityId VARCHAR(100) NOT NULL,
  action VARCHAR(60) NOT NULL, actorId VARCHAR(100) NOT NULL, requestId
VARCHAR(100) NOT NULL,
  timestamp DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3), details
JSON,
  INDEX audit_clinic (clinicId, timestamp)
);
CREATE TABLE IF NOT EXISTS prediction_models (
  doctorId VARCHAR(100) PRIMARY KEY, clinicId VARCHAR(100) NOT NULL, model
JSON NOT NULL,
  updatedAt DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
FOREIGN KEY (doctorId) REFERENCES doctors(id)
);
CREATE TABLE IF NOT EXISTS demo_seed (
  version VARCHAR(30) PRIMARY KEY, seededAt DATETIME(3) NOT NULL
DEFAULT CURRENT_TIMESTAMP(3)
);
CREATE OR REPLACE VIEW doctor_directory AS
SELECT d.*, c.name AS clinicName, c.district, s.name AS specialization,
```

```
  EXISTS(SELECT 1 FROM doctor_days dd WHERE dd.doctorId=d.id AND
dd.dayOfWeek=DAYNAME(CONVERT_TZ(UTC_TIMESTAMP(),'+00:00','+05:30')))
AND d.status='active' AS isAvailableToday,
  (SELECT JSON_ARRAYAGG(dayOfWeek) FROM doctor_days WHERE doctorId=d.id)
AS consultationDays,
  (SELECT JSON_ARRAYAGG(l.name) FROM doctor_languages dl JOIN languages l ON
l.id=dl.languageId WHERE dl.doctorId=d.id) AS languages,
  (SELECT JSON_ARRAYAGG(interest) FROM doctor_interests WHERE doctorId=d.id)
AS clinicalInterests
FROM doctors d JOIN clinics c ON c.id=d.clinicId JOIN specialties s ON s.id=d.specialtyId;
CREATE OR REPLACE VIEW visit_details AS
SELECT t.*, u.name AS patientName, u.phone AS patientPhone, d.name AS doctorName,
  d.clinicId, c.name AS clinicName
FROM queue_tokens t JOIN users u ON u.id=t.patientId JOIN doctors d ON d.id=t.doctorId
JOIN clinics c ON c.id=d.clinicId;
CREATE OR REPLACE VIEW appointment_details AS
SELECT a.*, u.name AS patientName, u.phone AS patientPhone, d.name AS doctorName,
s.name AS doctorSpecialization,
  d.clinicId, c.name AS clinicName, t.id AS tokenId
FROM appointments a JOIN users u ON u.id=a.patientId JOIN doctors d ON d.id=a.doctorId
JOIN clinics c ON c.id=d.clinicId JOIN specialties s ON s.id=d.specialtyId
LEFT JOIN queue_tokens t ON t.appointmentId=a.id;
CREATE OR REPLACE VIEW daily_patient_flow AS
SELECT d.clinicId, t.queueDate AS date, COUNT(*) AS total,
 SUM(t.status='completed') AS completed, SUM(t.status='cancelled') AS cancelled,
SUM(t.status='skipped') AS noShow
FROM queue_tokens t JOIN doctors d ON d.id=t.doctorId GROUP BY
d.clinicId,t.queueDate;
```

Appendix B Additional Code and Source Map

The transactional trigger demonstration is maintained in 004_transaction_demonstration.sql. Its disposable records are rolled back. The following excerpt is the actual cancellation and verification step.

```
UPDATE queue_tokens SET status='cancelled',updatedAt=UTC_TIMESTAMP(3) WHERE
id=@token;
-- AFTER UPDATE trigger synchronizes the appointment and releases the slot.
SELECT a.id,a.status appointmentStatus,t.status tokenStatus,s.bookedCount
FROM appointments a JOIN queue_tokens t ON t.appointmentId=a.id
JOIN appointment_slots s ON s.doctorId=a.doctorId AND s.slotDate=a.date AND
s.slotMinute=a.slotMinute WHERE a.id=@appointment;
SELECT action,actorId,details FROM audit_events WHERE entityId=@token ORDER BY
timestamp;
ROLLBACK;
SELECT (SELECT COUNT(*) FROM users WHERE id=@patient) remainingUsers,
 (SELECT COUNT(*) FROM appointments WHERE id=@appointment)
remainingAppointments,
 (SELECT COUNT(*) FROM queue_tokens WHERE id=@token) remainingTokens,
 (SELECT COUNT(*) FROM audit_events WHERE entityId=@token)
remainingAuditEvents;
```

Source file	**Purpose**
backend/sql/001_schema.sql	Relations, keys, checks, indexes and views
backend/sql/002_routines.sql	Function, two procedures and three triggers
backend/sql/003_demonstration_queries.sql	Manual search, aggregation and stored-object calls
backend/sql/004_transaction_demonstration.sql	Trigger synchronization and rollback verification
backend/src/dbms/connection.ts	Connection pool, transaction boundaries and error mapping
backend/src/repositories/mysqlRepo.ts	Parameterized queries, locking and persistence
backend/tests/mysql.test.ts	24 isolated SQL/API regression cases
src/pages/patient/DoctorList.tsx	SQL-backed directory filters and reconnect refresh
android/app/src/main/java/com/qureflow/dbms	Native navigation and discovery bridge

Appendix C Terminal Demonstration

With Docker Desktop and the MySQL container running, open a separate PowerShell window. This command can be run from any folder and prompts for the local demonstration database password.

```
docker exec -it qureflow-dbms-mysql-1 mysql -uqureflow -p qureflow_dbms
```

At the mysql prompt, execute each statement with a terminating semicolon. The client uses the database directly; no patient endpoint accepts arbitrary SQL.

```
SHOW TABLES;
DESCRIBE doctors;
SELECT COUNT(*) AS totalDoctors FROM doctors;
SELECT name,experienceYears,consultationFee
FROM doctors ORDER BY consultationFee LIMIT 10;
SELECT fn_consultation_minutes(
  '2026-10-19 10:00:00','2026-10-19 10:12:30'
) AS minutes;
SHOW TRIGGERS;
exit;
```

Appendix D Evidence Provenance

All web screenshots were captured from the running local application. SQL screenshots are browser-rendered views of actual executed query results; they are not photographs of a terminal window. Raw outputs, the browser booking record and the test log are retained under tmp/report/evidence for reproduction.

Tests and the rollback demonstration leave no disposable records. The browser workflow retains one cancelled synthetic booking: the visit count is therefore 6,196 rather than 6,195. The phone-sized image is a responsive browser view; native hotspot acceptance remains pending.